



Entregable — Líder del Proyecto

Integrante: Esteban Merlano **Roles:** Líder / Director del Proyecto · Backend Lead · Encargado del despliegue a producción **Proyecto:** Muelle Digital

1. Resumen ejecutivo

Muelle Digital es una plataforma web tipo marketplace que centraliza la oferta de experiencias ecoturísticas fluviales en Barrancabermeja (avistamiento de manatíes, pesca artesanal, paseos en canoa, aviturismo). Conecta a **turistas con guías y pescadores locales** verificados, permitiendo reservar, pagar en línea y calificar experiencias, con el **punto de encuentro geolocalizado en un mapa**.

Problema: el ecoturismo fluvial es informal: no hay un canal centralizado de reservas, ni precios claros, ni mecanismos de confianza, ni pagos digitales.

Solución entregada (MVP): catálogo con filtros, reservas con control de cupos, **pagos reales con Stripe (modo test)**, sistema de reseñas en tiempo real, verificación de guías por el administrador y panel de control total para el admin.

2. Objetivos y alcance

Objetivo general: desarrollar una plataforma fullstack que formalice y organice el ecoturismo fluvial mediante reservas y pagos digitales.

Objetivos específicos cumplidos:

1. ☒ Reservas digitales con control de cupos.
2. ☒ Pagos en línea (Stripe + Nequi/Daviplata simulados).
3. ☒ Sistema de reputación (reseñas 1–5 en tiempo real).
4. ☒ Geolocalización del punto de encuentro (mapas).
5. ☒ Formalización: verificación de guías con aprobación del admin.

3. Plan de proyecto — Cronograma con hitos

```
gantt      title Muelle Digital — Cronograma (8 semanas)
dateFormat YYYY-MM-DD      section Definición      Historias de
usuario y roles      :done, s1, 2026-04-06, 7d      section
Arquitectura      Modelo de datos + Figma + Stack      :done, s2,
2026-04-13, 7d      section Sprints      Sprint 1: Login + DB +
base      :done, s3, 2026-04-20, 7d      Sprint 2: Experiencias +
reservas :done, s4, 2026-04-27, 7d      Sprint 3: Reseñas + pagos
+ mapas :done, s5, 2026-05-04, 7d      Sprint 4: Panel admin + UI
:done, s6, 2026-05-11, 7d      section Calidad      QA + Code
Review      :done, s7, 2026-05-18, 7d      section
```

Entrega Documentación + despliegue + demo :active, s8, 2026-05-25, 7d

Semana	Hito	Dependencia	Estado
1	Historias de usuario, roles, repositorio	—	✓
2	Diagrama ER, prototipo, stack definido	Semana 1	✓
3	Login + conexión a NeonDB + estructura	Semana 2	✓
4	CRUD de experiencias + reservas	Semana 3	✓
5	Reseñas en tiempo real + pagos + mapas	Semana 4	✓
6	Panel admin + verificación de guías + UI	Semana 5	✓
7	Pruebas funcionales y de seguridad + Code Review	Semana 6	✓
8	README, manual, despliegue y sustentación	Semana 7	📦

4. Asignación de recursos y responsabilidades

Persona	Rol	Responsabilidad principal	Herramientas
Esteban Merlano	Líder + Backend + Deploy	API, autenticación, pagos, despliegue, gestión	Node/Express, Stripe, Vercel, ClickUp

Keyner Trujillo	Frontend	Pantallas, UX, consumo de API	React, Vite, Tailwind
Andres Rangel	Base de Datos	Modelo ER, diccionario, seguridad de datos	PostgreSQL, NeonDB
Jhon Escobar	QA	Pruebas funcionales/seguridad, reporte de bugs	Postman, checklist, Git PRs

Matriz RACI (resumen): el Líder es Responsable/Aprobador de la integración; cada rol es Responsable de su área; todos Consultados en decisiones de arquitectura.

5. Análisis de riesgos y mitigación

#	Riesgo	Prob.	Impacto	Mitigación
R1	Fuga de datos sensibles (credenciales)	Media	Alto	Contraseñas con bcrypt, JWT, claves en <code>.env</code> (no en Git), acceso por rol
R2	Caída del servicio de mapas gratuito	Baja	Medio	MapLibre permite cambiar de proveedor de tiles en 1 línea
R3	Costos inesperados de infraestructura	Baja	Medio	Uso de planes gratuitos (Neon, Vercel, OpenFreeMap, Stripe test)
R4	Integración de pagos falla	Media	Alto	Confirmación doble (webhook + verificación de sesión); modo test

R5	Desalineación del equipo	Media	Medio	Daily de 15 min, ClickUp, PRs obligatorios
R6	Pérdida de código	Baja	Alto	Git + GitHub, ramas protegidas (GitFlow)

6. Presupuesto estimado

Proyecto académico con infraestructura en **planes gratuitos**. Se estima el costo equivalente de mercado.

Concepto	Detalle	Costo (académico)	Costo equivalente mercado
Mano de obra	4 personas × 8 semanas × ~8 h/sem ≈ 256 h	\$0 (estudiantes)	~\$6.400.000 COP (≈ \$25k COP/h)
Base de datos	NeonDB free tier	\$0	~\$0–\$19 USD/mes
Hosting	Vercel Hobby	\$0	~\$0–\$20 USD/mes
Mapas	OpenFreeMap / OSM	\$0	\$0
Pasarela de pago	Stripe (test)	\$0	2.9% + comisión por transacción (prod)
Dominio (opcional)	.com	\$0 (subdominio Vercel)	~\$12 USD/año
Total proyecto		\$0	≈ \$6.4M COP (valor del trabajo)

7. Documentación de gestión (software de gestión)

- **Herramienta:** ClickUp (tablero Kanban). Configuración, columnas y **entregables por semana** en [05-CLICKUP-entregables-semanales.md](#).

- **Métricas de seguimiento:** n° de tarjetas por estado, cumplimiento de hitos por semana, % de HU completadas (13/13).
- **Reportes de estado:** al cierre de cada sprint (qué se hizo, qué sigue, bloqueos).
- **Registro de decisiones importantes:**
 - Backend en **Node/Express** (mismo lenguaje que el front, gratis, fácil de desplegar en Vercel).
 - **PostgreSQL/Neon** sobre Mongo (datos relacionales y normalizados para un marketplace).
 - **Sin ORM** (SQL parametrizado con pg) para evitar sobreingeniería.
 - **Stripe Checkout** (hosted) en vez de formularios de tarjeta propios (seguridad/PCI).

8. Mi trabajo técnico — Backend (API REST)

Tecnologías: Node.js + Express (ESM), pg (PostgreSQL), jsonwebtoken, bcryptjs, stripe, cors, dotenv.

Arquitectura por capas:

```
backend/src/ |— app.js / index.js      ← app Express y arranque
|— config/          ← db.js (pool PostgreSQL), stripe.js
|— middleware/auth.js ← verificación JWT + control por rol
|— controllers/      ← lógica (auth, experiencias,
reservas, ratings, pagos, solicitudes, admin) |— routes/
← definición de endpoints
```

Decisiones de seguridad:

- Contraseñas con **bcrypt** (hash + salt), nunca en texto plano.
- **JWT** con expiración; middleware `requireAuth` y `requireRole`.
- El registro **siempre** crea rol turista; el rol guía solo lo otorga el admin (no se puede auto-asignar).
- Claves secretas (DB, JWT, Stripe) en **.envfuera de Git**.

Pagos (Stripe): al crear una experiencia se genera `Product+Price`; al reservar se crea una `Checkout Session`; la confirmación es **idempotente** vía `webhook` y verificación de sesión. Diseñado para escalar a **comisiones por guía** (Stripe Connect) — ya se guarda comision por pago.

Documentación de la API (Swagger): la API está documentada con **OpenAPI 3 + Swagger UI**, accesible de forma interactiva en `/api/docs` (y el JSON en `/api/openapi.json`). Cumple el entregable "guía de la API (Swagger)" de la Semana 8.

Detalle completo de endpoints y arquitectura en `MUELLE_DIGITAL_MASTER.md` §14 y §21.

9. Mi trabajo técnico — Despliegue a producción

Objetivo: todo en la nube y gratis.

- **Base de datos:** NeonDB (PostgreSQL serverless, endpoint pooler ideal para serverless).
- **Backend** → **Vercel** como Serverless Function (backend/api/index.js + backend/vercel.json).
- **Frontend** → **Vercel** (Vite estático + fallback SPA en frontend/vercel.json).
- **Ambientes:** desarrollo (local), producción (Vercel). Variables de entorno por ambiente (DATABASE_URL, JWT_SECRET, STRIPE_*, CLIENT_URL).
- **CI/CD:** despliegue automático de Vercel en cada push a main. (Opcional: GitHub Actions para tests.)
- **Monitoreo:** logs de Vercel + healthcheck GET /api/health.

10. Lecciones aprendidas y recomendaciones

- **Acierto:** elegir un stack homogéneo (JS en front y back) aceleró el desarrollo.
- **Acierto:** confirmación doble de pagos hizo el sistema robusto incluso sin webhook en local.
- **Mejora:** automatizar pruebas (Jest/Playwright) y documentar la API con Swagger.
- **Recomendación:** migrar a Stripe Connect para repartir pagos a cada guía.



Guion de presentación (3-5 min)

1. **(30s)** "Soy Esteban, líder del proyecto y encargado del backend y el despliegue."
2. **(2 min)** Muestro el **plan/cronograma** y el **tablero de ClickUp**; explico la arquitectura del backend y cómo se despliega en Vercel + Neon.
3. **(1 min)** Cómo coordiné al equipo (roles, dailies, PRs) y cómo integré el trabajo de todos.
4. **(30s)** Preguntas: decisiones técnicas (por qué Node, por qué Postgres, seguridad de claves).